

Programmation Orientée Objet

— *Concepts* —

Cédric Buche

École Nationale d'Ingénieurs de Brest (ENIB)

28 janvier 2014



Objectifs thématiques de POO

Que va m'apprendre POO ?

- **Le modèle objet :**
 - Objet
 - Classe
 - Encapsulation
 - Collaboration
 - Héritage
 - (Abstraction)
 - (Modularité)
 - (Persistance)
- **Modéliser objet**
 - La partie statique du langage UML :
 - Diag. de cas d'utilisation
 - Diag. de classes
 - Modèle d'interaction (Diag. séquence + communication)

Et la suite ... ?

Cycle Ingénieur ENIB - Tronc commun

Semestre	Thème	Volume
S5	Prog. par objets (Java)	45h
S6	Prog. par objets (C++)	45h
S6	Modèles pour l'ingénierie des systèmes (UML)	45h
S6	Base de données	22h
S7	Réseaux	90h
S8	Stage en entreprise	45h
S10	Stage en entreprise	45h

Équipe pédagogique 2013

Enseignants-chercheurs

BOSSER	Anne-Gwen	Cours + Labo	boss@enib.fr
DESMEULLES	Gireg	Cours + Labo	desmeulles@enib.fr

Planning

Module : <i>Programmation Orientée Objet — POO</i>	2013
Responsable : Cédric BUCHE (CERV, bureau 30, tel : 02 98 05 89 66, email : buche@enib.fr)	S4
Intervenants : C. BUCHE (cb)	

Semaine	LABO	CTD
37		<i>POO — Justification modèle objet</i> <i>ctd</i> cb
37		<i>POO — Concepts fondateurs</i> <i>ctd</i> cb
38	<i>POO — Rappels python</i> <i>labo</i> cb	
38	<i>POO — Concepts fondateurs</i> <i>labo</i> cb	
39		<i>POO — Encapsulation</i> <i>ctd</i> cb
39		<i>POO — Collaboration et hiérarchie</i> <i>ctd</i> cb
40	<i>POO — Encapsulation</i> <i>labo</i> cb	
40	<i>POO — Collaboration et héritage</i> <i>labo</i> cb	
41		<i>UML — Généralités</i> <i>ctd</i> cb
41		<i>UML — Classes</i> <i>ctd</i> cb
42	<i>UML — Classes</i> <i>labo</i> cb	
42	<i>UML — Classes</i> <i>labo</i> cb	
43		<i>UML — Classes</i> <i>ctd</i> cb
43		<i>UML — Classes</i> <i>ctd</i> cb
44	<i>Vacances</i> <i>Toussaint</i>	

Semaine	LABO	CTD
45	<i>UML — Classes</i> <i>labo</i> cb	
45	<i>UML — Classes</i> <i>labo</i> cb	
46		<i>UML — Use Case (diag.)</i> <i>ctd</i> cb
46		<i>UML — Use Case (scénarios)</i> <i>ctd</i> cb
47	<i>UML — Use Case (diag.)</i> <i>labo</i> cb	
47	<i>UML — Use Case (scénarios)</i> <i>labo</i> cb	
48		<i>UML — Interactions (com)</i> <i>ctd</i> cb
48		<i>UML — Interactions (seq)</i> <i>ctd</i> cb
49	<i>UML — Interactions (com)</i> <i>labo</i> cb	
49	<i>UML — Interactions (seq)</i> <i>labo</i> cb	
50		
50		
51		
51		

Types de contrôle

Contrôle d'attention

- Contrôle de l'attention sur un cours particulier - QCM
- Durée 5-10'
- En fin de cours

Contrôle des connaissances

- Contrôle des connaissances acquis sur un thème - QCM
- Durée 5-10'
- En début de labo

Contrôle d'attention + Contrôle des connaissances
→ Note de travail

Types de contrôle

Contrôle d'attention

- Contrôle de l'attention sur un cours particulier - QCM
- Durée 5-10'
- En fin de cours

Contrôle des connaissances

- Contrôle des connaissances acquis sur un thème - QCM
- Durée 5-10'
- En début de labo

Contrôle d'attention + Contrôle des connaissances
→ Note de travail

Types de contrôle

Contrôle d'attention

- Contrôle de l'attention sur un cours particulier - QCM
- Durée 5-10'
- En fin de cours

Contrôle des connaissances

- Contrôle des connaissances acquis sur un thème - QCM
- Durée 5-10'
- En début de labo

Contrôle d'attention + Contrôle des connaissances
→ Note de travail

Exemple de QCM

Informatique

Contrôle Continu

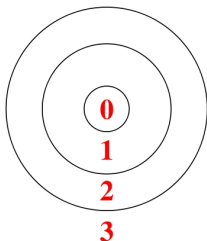
Nom / Prenom :

Groupe :

1. QCM titre 4 _____
 - (a) réponse fausse 4.2 
 - (b) réponse fausse 4.1 
 - (c) réponse fausse 4.5 
 - (d) réponse juste 4 
 - (e) réponse fausse 4.3 
 - (f) réponse fausse 4.4 
2. QCM titre 1 _____
 - (a) réponse juste 1 
 - (b) réponse fausse 1.2 
 - (c) réponse fausse 1.3 
 - (d) réponse fausse 1.1 
3. QCM titre 2 _____
 - (a) réponse juste 2 
 - (b) réponse fausse 2.1 
 - (c) réponse fausse 2.2 
4. QCM titre 3 _____
 - (a) réponse juste 3 
 - (b) réponse fausse 3.1 
 - (c) réponse fausse 3.2 

Notation QCM

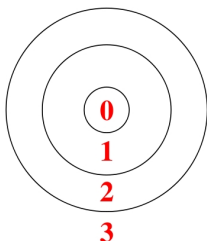
Note : distance au but



0 :	dans le mille!	→	objectif atteint
1 :	pas mal	→	objectif en vue
2 :	tout juste sur la cible	→	objectif lointain
3 :	même pas touché	→	objectif pas atteint

Notation QCM

Note : distance au but



0 :	dans le mille!	→	objectif atteint
1 :	pas mal	→	objectif en vue
2 :	tout juste sur la cible	→	objectif lointain
3 :	même pas touché	→	objectif pas atteint

Contrôle de synthèse

- Contrôle des compétences acquises sur un demi-semestre
- Durée 1h20
 - Concepts objet
 - UML

Évaluation des apprentissages

Et si je suis absent ?

- à un "Contrôle d'attention"
 - Je justifie mon absence (certificat médical)
- à un "Contrôle des connaissances"
 - Je justifie mon absence (certificat médical)
- à un "Contrôle de synthèse"
 - Je justifie mon absence (certificat médical)
 - Rattrapage

Plan de la présentation

- 1 Justification du modèle objet (1 UC)
- 2 Concepts fondateurs (1 UC)
- 3 Encapsulation (1 UC)
- 4 Hiérarchie (1 UC)
- 5 Collaborations entre objets (1 UC)
- 6 Pour aller plus loin

- 1 Justification du modèle objet (1 UC)
- 2 Concepts fondateurs (1 UC)
- 3 Encapsulation (1 UC)
- 4 Hiérarchie (1 UC)
- 5 Collaborations entre objets (1 UC)
- 6 Pour aller plus loin

Cours 1 / Labo 2

Objectifs

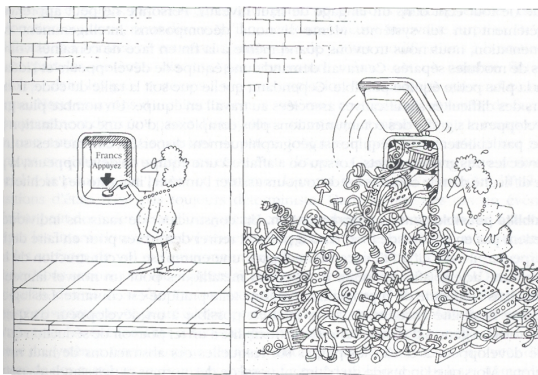
- Justifier le modèle objet
- Notion de classe :
 - définition
 - constructeur
- Notion d'objet :
 - instanciation
 - manipulation

Complexité des problèmes

« La complexité des logiciels est une caractéristique innée et non propriété accidentelle » [Brooks, 1987]



Difficulté du contrôle du processus de développement



La tâche de l'équipe de développement de logiciel est de donner l'illusion de la simplicité [Booch, 1992]

Conséquences d'une complexité sans limites

« *Plus un système est complexe, plus il est susceptible d'effondrement* » [Peter, 1986]



Est ce qu'un entrepreneur songe à ajouter une nouvelle assise à un immeuble de 100 étages existant ? Ce genre de considérations est pourtant demandé par certains utilisateurs de logiciels

Qualité du logiciel

Extensibilité :

faculté d'adaptation d'un logiciel aux changements de spécification.

- **simplicité de la conception**
- **décentralisation**

Réutilisabilité :

aptitude d'un logiciel à être réutilisé en tout ou en partie pour de nouvelles applications.

Ne pas réinventer la roue !

Programmer *moins* pour programmer *mieux* !

Qualité du logiciel

Extensibilité :

faculté d'adaptation d'un logiciel aux changements de spécification.

- **simplicité de la conception**
- **décentralisation**

Réutilisabilité :

aptitude d'un logiciel à être réutilisé en tout ou en partie pour de nouvelles applications.

Ne pas réinventer la roue !

Programmer *moins* pour programmer *mieux* !

Qualité du logiciel

Extensibilité :

faculté d'adaptation d'un logiciel aux changements de spécification.

- **simplicité de la conception**
- **décentralisation**

Réutilisabilité :

aptitude d'un logiciel à être réutilisé en tout ou en partie pour de nouvelles applications.

Ne pas réinventer la roue !

Programmer *moins* pour programmer *mieux* !

Paradigme de programmation procédurale

Programmation procédurale

- séparation entre code et données.

Variables (globales)

- contiennent vos données
- sont utilisées par des fonctions/procédures.

Fonctions

- peuvent modifier une variable
- peuvent passer cette variable à une autre fonction/procédure.

Paradigme de programmation procédurale

Programmation procédurale

- séparation entre code et données.

Variables (globales)

- contiennent vos données
- sont utilisées par des fonctions/procédures.

Fonctions

- peuvent modifier une variable
- peuvent passer cette variable à une autre fonction/procédure.

Paradigme de programmation procédurale

Programmation procédurale

- séparation entre code et données.

Variables (globales)

- contiennent vos données
- sont utilisées par des fonctions/procédures.

Fonctions

- peuvent modifier une variable
- peuvent passer cette variable à une autre fonction/procédure.

Exemple

Dans une entreprise située à Biarritz,
Robert (num de secu 10573123456),
est employé sur un poste de technicien

```
1  #####
2  # Version 1
3  #   variables globales
4  #####
5
6  num_secu_robert      = 10573123456
7  nom_robert          = "Robert"
8  qualification_robert = "Technicien"
9  lieu_de_travail_robert = "Biarritz"
```

Limites de la programmation procédurale : Exemple

Ajout d'un employé bernard

```
1  # robert
2  num_secu_robert      = 10573123456
3  nom_robert           = "Robert"
4  qualification_robert = "Technicien"
5  lieu_de_travail_robert = "Biarritz"
6
7  # bernard
8  num_secu_bernard     = 2881111001
9  nom_bernard          = "Bernard"
10 qualification_bernard = "Ingenieur"
11 lieu_de_travail_bernard = "Biarritz"
12
13 # ...
```

Limites de la programmation procédurale

```
1  #####
2  # Version 2
3  # utilisation de conteneurs
4  #####
5
6  tab_robert=[10573123456,
7              "robert",
8              "Technicien",
9              "Biarritz"]
10 tab_mickey=[14456464443,
11             "mickey",
12             "Ingenieur",
13             "Brest"]
14 tab=[]
15 tab.append(tab_robert)
16 tab.append(tab_mickey)
17 print tab[0][0]                # num secu robert
```

Vers la programmation orientée objet

Définition de la structure de donnée par la notion de **classe**

```
1  #####
2  # Version 3
3  #   structuration des donnees
4  #####
5
6  class Employe :
7      num_secu
8      nom
9      qualification
10     Lieu_de_travail
```

Vers la programmation orientée objet : classe

Classe

- déclare les propriétés communes d'un *ensemble* d'objets

Exemple

La classe Voiture possède les propriétés suivantes (les **attributs**) :

- couleur
- puissance

Vers la programmation orientée objet : classe

Classe

- déclare les propriétés communes d'un *ensemble* d'objets

Exemple

La classe **Voiture** possède les propriétés suivantes (les **attributs**) :

- couleur
- puissance

Vers la programmation orientée objet : objets

Classe → Objets

- usine à partir de laquelle il est possible de créer des objets.

Objets : exemple instance de la classe Voiture

- Voiture "Clio 2007 version roland garros"
 - couleur: verte
 - puissance: 70 Ch
- Voiture "307 blue lagoon"
 - couleur: bleue
 - puissance: 90 Ch

Vers la programmation orientée objet : objets

Classe → Objets

- usine à partir de laquelle il est possible de créer des objets.

Objets : exemple instance de la classe Voiture

- Voiture "Clio 2007 version roland garros"
 - couleur: verte
 - puissance: 70 Ch
- Voiture "307 blue lagoon"
 - couleur: bleue
 - puissance: 90 Ch

En pratique ...

Mise en place d'**objets** concrets (*instances* de classe)

```
1  # robert
2  e1=Employe()      # objet e1 : instance de la classe Employe
3  e1.num_secu       = 10573123456
4  e1.nom            = "Robert"
5  e1.qualification  = "Technicien"
6  e1.Lieu_de_travail = "Biarritz"
7
8  print (e1.num_secu ,
9         e1.nom ,
10        e1.qualification ,
11        e1.Lieu_de_travail)
12
13 # mickey
14 e2=Employe()      # objet e2 : instance de la classe Employe
15 e2.num_secu       = 14456464443
16 e2.nom            = "Mickey"
17 e2.qualification  = "Inge"
18 e2.Lieu_de_travail = "Brest"
```

En pratique ...

```
1  # norbert
2  e3=Employe()
3
4  print (e3.num_secu,          # ERREUR
5         e3.nom,
6         e3.qualification,
7         e3.Lieu_de_travail)
```

Vers la programmation orientée objet : Le constructeur

Constructeur

- les attributs doivent être initialisés avec une valeur « par défaut »
- appel à une fonction particulière : le **constructeur**

En pratique ...

```
1
2  class Employe:
3      def __init__(self):          # Constructeur de la classe Employe
4          self.num_secu           = 0000000000
5          self.nom                 = "no_name"
6          self.qualification       = "novice"
7          self.Lieu_de_travail    = "Paris"
```



En python, le premier argument du constructeur est toujours **self**

En pratique ...

```
1  # norbert
2  e3=Employe()      # creation de e3 : appel implicite de init de Employe
3
4  print (e3.num_secu,      # OK !!!
5          e3.nom,
6          e3.qualification,
7          e3.Lieu_de_travail)
```

En pratique ...

```
1
2  class Employe:
3      def __init__( self ,
4                    le_num_secu ,
5                    le_nom ,
6                    la_qualification ,
7                    le_Lieu_de_travail ):
8
9      self.num_secu      = le_num_secu
10     self.nom            = le_nom
11     self.qualification  = la_qualification
12     self.Lieu_de_travail = le_Lieu_de_travail
```

En pratique ...



En python,

→ `--init--` appelée implicitement à la création d'un objet

→ le constructeur peut posséder plusieurs arguments (en plus de `self`)

Vers la programmation orientée objet : Le constructeur

```
1
2     class Employe:
3         def __init__( self ,
4                       le_num_secu      = 000000000 ,
5                       le_nom          = "no_name" ,
6                       la_qualification = "novice" ,
7                       le_Lieu_de_travail = "Paris" ):
8
9             self.num_secu      = le_num_secu
10            self.nom            = le_nom
11            self.qualification   = la_qualification
12            self.Lieu_de_travail = le_Lieu_de_travail
13
14
15     ##### Programme Principal #####
16
17     e1 = Employe( le_Lieu_de_travail = "Brest" )
```

Plan de la présentation

- 1 Justification du modèle objet (1 UC)
- 2 Concepts fondateurs (1 UC)
- 3 Encapsulation (1 UC)
- 4 Hiérarchie (1 UC)
- 5 Collaborations entre objets (1 UC)
- 6 Pour aller plus loin

Cours 2 / Labo 2

Objectifs

- Classe :
 - Attributs
 - Méthodes
 - Constructeur / destructeur
- Objet :
 - État
 - Comportement
 - Identité

Cours 2 / Labo 2

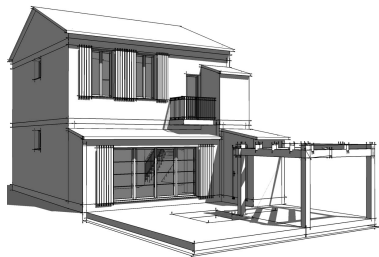
Objectifs

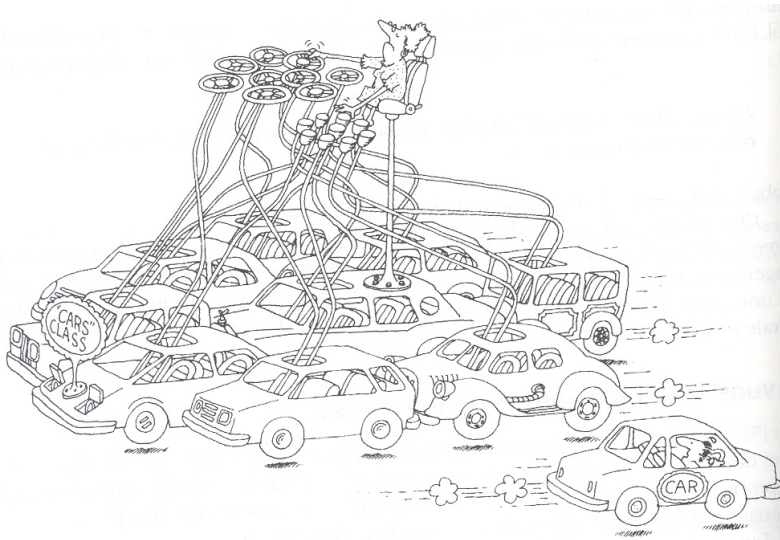
- Classe :
 - Attributs
 - Méthodes
 - Constructeur / destructeur
- Objet :
 - État
 - Comportement
 - Identité

Les classes

Je croyais qu'on allait apprendre à créer des objets,
pourquoi créer une classe ?

- pour créer un objet, il faut d'abord créer une classe !
- exemple : pour construire une maison,
 - besoin d'un plan d'architecte
 - classe = plan,
 - l'objet = maison.
 - "Créer une classe", c'est dessiner les plans de l'objet.





Une **classe** représente un **ensemble d'objets** qui partagent une structure commune et un comportement commun [Booch, 1992]

Les classes

- Une classe est la description d'une famille d'objets qui ont la même structure et les mêmes comportements.
- Une classe est une sorte de moule à partir duquel sont générés les objets.



Une classe

génération
→

des objets

Les classes

- Une classe est la description d'une famille d'objets qui ont la même structure et les mêmes comportements.
- Une classe est une sorte de moule à partir duquel sont générés les objets.



Une classe

génération →

des objets

Les classes

- Une classe est constituée :
 - ① d'un ensemble de variables (ou *attributs*) qui décrivent la structure des objets ;
 - ② d'un ensemble de fonctions (ou *méthodes*) qui sont applicables aux objets, et qui décrivent leur comportement.



attributs : variables contenues dans la classe,
on parle aussi de "variables membres"

méthodes : fonctions contenues dans la classe,
on parle aussi de "fonctions membres"
ou de compétences
ou de services

Les classes

- Une classe est constituée :
 - ① d'un ensemble de variables (ou *attributs*) qui décrivent la structure des objets ;
 - ② d'un ensemble de fonctions (ou *méthodes*) qui sont applicables aux objets, et qui décrivent leur comportement.



attributs : variables contenues dans la classe,
on parle aussi de "variables membres"

méthodes : fonctions contenues dans la classe,
on parle aussi de "fonctions membres"
ou de compétences
ou de services

Classe — Objets

① Classe

- Abstraction
- Définit une infinité d'objets instances

② Objet = état + comportement + identité

- Attributs
- Méthodes
- Identité



On dit qu'un objet est une **instance** d'une classe

Classe — Objets

- ① Classe
 - Abstraction
 - Définit une infinité d'objets instances
- ② Objet = état + comportement + identité
 - Attributs
 - Méthodes
 - Identité



On dit qu'un objet est une *instance d'une classe*

Classe — Objets

① Classe

- Abstraction
- Définit une infinité d'objets instances

② Objet = état + comportement + identité

- Attributs
- Méthodes
- Identité



On dit qu'un objet est une **instance d'une classe**

État

État d'un objet

- regroupe les valeurs instantanées de tous ses **attributs**
- évolue au cours du temps
- est la conséquence de ses comportements passés

État ... En pratique

```
1  ##### definition classe #####
2  class Voiture:
3      def __init__(self, v1="nomarque", v2="nocolor", v3=0):
4          self.marque = v1
5          self.couleur = v2
6          self.reserveEssence = v3
7
8  ### Objet #####
9  homer_mobile = Voiture()
10
11      ## Etat
12  homer_mobile.marque      = "pipo"
13  homer_mobile.couleur     = "rose"
14  homer_mobile.reserveEssence = 30
```



Attributs et variables

- les attributs caractérisent l'état des instances d'une classe. Ils participent à la modélisation du problème.
- les variables sont des mémoires locales à des méthodes. Elles sont là pour faciliter la gestion du traitement.
- l'accès aux variables est limité au bloc où elles sont déclarées : règle de portée

Attributs et variables

- les attributs caractérisent l'état des instances d'une classe. Ils participent à la modélisation du problème.
- les variables sont des mémoires locales à des méthodes. Elles sont là pour faciliter la gestion du traitement.
- l'accès aux variables est limité au bloc où elles sont déclarées : règle de portée

Attributs et variables

- les attributs caractérisent l'état des instances d'une classe. Ils participent à la modélisation du problème.
- les variables sont des mémoires locales à des méthodes. Elles sont là pour faciliter la gestion du traitement.
- l'accès aux variables est limité au bloc où elles sont déclarées : règle de portée

Comportement

Comportement d'un objet

- regroupe toutes ses compétences
 - décrit les actions et les réactions d'un objet
- se représente sous la forme de **méthodes**
(appelées parfois fonctions membres)

Pour un objet "homer_mobile"

- démarrer
- arrêter
- freiner
- ...



Comportement

Comportement d'un objet

- regroupe toutes ses compétences
 - décrit les actions et les réactions d'un objet
- se représente sous la forme de **méthodes**
(appelées parfois fonctions membres)

Pour un objet "homer_mobile"

- démarrer
- arrêter
- freiner
- ...



Comportement ... En pratique

```
1  ##### definition classe #####
2  class Voiture:
3      def __init__(self, v1="nomarque", v2="nocolor", v3=0):
4          self.marque = v1
5          self.couleur = v2
6          self.reserveEssence = v3
7
8      def demarrer(self):
9          print "je démarre"
10     def arreter(self):
11         print "je m'arrete"
12     ...
13     ### Objet #####
14     homer_mobile = Voiture()
15
16     ## Comportement
17     homer_mobile.demarrer()
18     homer_mobile.arreter()
```



Comportement — État

L'état et le comportement sont liés

```
homer_mobile.démarrer()
```

ne marchera pas si

```
homer_mobile.reserveEssence  
== 0
```



Comportement — État

```
1  ##### definition classe #####
2  class Voiture:
3      def __init__(self, v1="nomarque", v2="nocolor", v3=0):
4          self.marque = v1
5          self.couleur = v2
6          self.reserveEssence = v3
7
8      def demarrer(self):
9          if self.reserveEssence > 0 :
10             print "je démarre"
11          else :
12             print "je ne peux pas démarrer"
```

État : évolution

Pour l'objet "homer_mobile"

- marque : pipo
- couleur : rose
- réserve d'essence :
30l



Après 100 kms ...

- marque : pipo
- couleur : rose
- réserve d'essence :
25l



État : évolution

Pour l'objet "homer_mobile"

- marque : pipo
- couleur : rose
- réserve d'essence :
30l



Après 100 kms ...

- marque : pipo
- couleur : rose
- réserve d'essence :
25l



État : évolution

```
1  ##### definition classe #####
2  class Voiture:
3      def __init__(self, v1="nomarque", v2="nocolor", v3=0):
4          self.marque = v1
5          self.couleur = v2
6          self.reserveEssence = v3
7
8      def demarrer(self):
9          if self.reserveEssence > 0 :
10             print "je démarre"
11          else :
12             print "je ne peux pas demarrer"
13
14      def rouler(self, nbKm):
15          self.reserveEssence = self.reserveEssence - 5*nbKm/100;
16
17
18  #####
19  homer_mobile = Voiture()
20  homer_mobile.reserveEssence = 30
21  homer_mobile.demarrer()
22  homer_mobile.rouler(100)
23  print homer_mobile.reserveEssence
```

Identité

L'identité

- est propre à l'objet et le caractérise
- permet de distinguer tout objet indépendamment de son état



Voiture.1



Voiture.2

Identité

L'identité

- est propre à l'objet et le caractérise
- permet de distinguer tout objet indépendamment de son état



Voiture.1



Voiture.2

Identité

→ nommer chaque objet
(ex. attribut "name")
`print homer_mobile.name`



Voiture.1
0xb7d4e0ec



Voiture.2
0xb7d53eec

→ leur référence
`print homer_mobile`

Le chaos des objets



Attention, ne pas oublier !!

- Soient 3 objets :

- même structures de données (attributs)
- même comportement (méthodes)

→ Il faut les décrire de façon abstraite : une **classe**

Construction

- A la déclaration d'un objet, les attributs ne sont pas initialisés.
- Cette phase d'initialisation des attributs est effectuée par une méthode particulière : le constructeur.
- Le constructeur est appelé lors de la construction de l'objet.
- Une classe peut définir son(ses) constructeur(s) avec des paramètres différents

Construction

- A la déclaration d'un objet, les attributs ne sont pas initialisés.
- Cette phase d'initialisation des attributs est effectuée par une méthode particulière : le constructeur.
- Le constructeur est appelé lors de la construction de l'objet.
- Une classe peut définir son(ses) constructeur(s) avec des paramètres différents

Construction

- A la déclaration d'un objet, les attributs ne sont pas initialisés.
- Cette phase d'initialisation des attributs est effectuée par une méthode particulière : le constructeur.
- Le constructeur est appelé lors de la construction de l'objet.
- Une classe peut définir son(ses) constructeur(s) avec des paramètres différents

Construction

- A la déclaration d'un objet, les attributs ne sont pas initialisés.
- Cette phase d'initialisation des attributs est effectuée par une méthode particulière : le constructeur.
- Le constructeur est appelé lors de la construction de l'objet.
- Une classe peut définir son(ses) constructeur(s) avec des paramètres différents

En pratique ...

```
1
2  class Complexe:
3      def __init__(self, r=0.0, i=0.0):
4          self.reel = r
5          self.img = i
6
7
8
9  ##### Programme principal #####
10 c1 = Complexe()
11 c2 = Complexe(0.5)
12 c3 = Complexe(0.5,0.2)
13 c4 = Complexe(i=0.2)
```

Destruction

- Méthode spéciale appelée lors de la suppression d'une instance : le destructeur de la classe
- Objectif : détruire tous les objets créés par l'objet courant au cours de son existence



En python,

→ la définition d'un destructeur n'est pas obligatoire

Destruction

- Méthode spéciale appelée lors de la suppression d'une instance : le destructeur de la classe
- Objectif : détruire tous les objets créés par l'objet courant au cours de son existence



En python,
→ la définition d'un destructeur n'est pas obligatoire

Destruction

- Méthode spéciale appelée lors de la suppression d'une instance : le destructeur de la classe
- Objectif : détruire tous les objets créés par l'objet courant au cours de son existence



En python,

→ la définition d'un destructeur n'est pas obligatoire

En pratique ...

```
1
2 class Complexe:
3     def __del__(self):
4         print "I 'm dying!"
```



En python,

→ `--del--` appelée implicitement à la destruction d'un objet

Garbage collector

Appel du destructeur ?

- dans certains langages, un programme s'exécute en tâche de fond afin de supprimer la mémoire associée à un objet
- le ramasse miettes ou "garbage collector"

En pratique ...

```
1  class Test:
2      def __del__(self):
3          print "I am dying"
4
5
6
7  >>> t1 = Test()    # une instance de Test est cree
8  >>> t1 = None      # t1 ne permet plus d'accéder à l'instance
9  I am dying         # elle est susceptible d'être détruite ce qui
10 >>>                # arrive instantanément ici mais n'est pas garanti
```

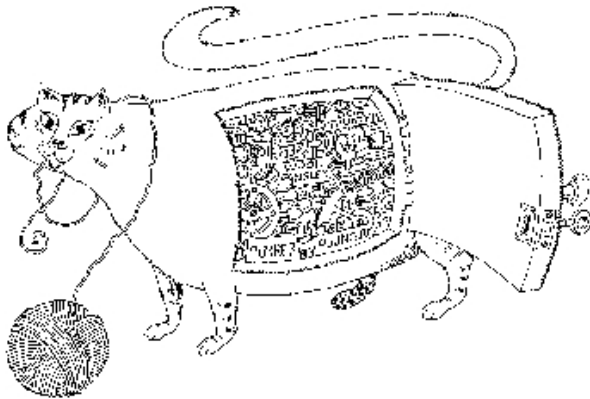
Plan de la présentation

- 1 Justification du modèle objet (1 UC)
- 2 Concepts fondateurs (1 UC)
- 3 Encapsulation (1 UC)**
- 4 Hiérarchie (1 UC)
- 5 Collaborations entre objets (1 UC)
- 6 Pour aller plus loin

Cours 3 / Labo 3

Objectifs

- Encapsulation
- Règles d'encapsulation



L'encapsulation occulte les détails de l'implémentation d'un objet
[Booch, 1992]

Contrôle d'accès : illustration

Gestion de compte en banque

- classe `Compte`
 - avec un attribut `solde`

```
1  class Compte:
2      def __init__(self):
3          self.solde = 0          # solde nul
4
5      def affiche(self):
6          print "Le solde de votre compte est de " self.solde
7
8
9  ##### Programme principal #####
10 c = Compte()
11 c.affiche()          # pourquoi pas
12 c.solde = 1000000    # ben tiens !
```

Contrôle d'accès

- Lors de la définition d'une classe il est possible de restreindre voire d'interdire l'accès (aux objets des autres classes) à des attributs ou méthodes des instances de cette classe.



Contrôle d'accès

- Modificateurs d'accès lors de la déclaration d'attributs ou méthodes :
 - **privée** accessible uniquement depuis des instances de la classe
 - **public** accessible par tout le monde
(ie. tous ceux qui possèdent une référence sur l'objet)

→ Rendre `solde` privé

En pratique ...

```
1  class Test:
2
3      def __init__(self, laValeurPublic, laValeurPrivee):
4          self.valeurPublic = laValeurPublic
5          self.__valeurPrivee = laValeurPrivee
6
7
8      def f1(self):
9          ...
10
11     def __f2(self):
12         ...
13
14     ##### Programme principal #####
15     t1 = Test()
16
17     print t1.valeurPublic          # OK
18     print t1.__valeurPrivee       # ERREUR
19
20     t1.f1()                       # OK
21     t1.__f2()                     # ERREUR
```

Contrôle d'accès

Le programmeur créateur peut modifier le comportement sans impact :

- contrôle lors de la modification

```
1 def debit(self, laValeur):
2
3     if self.__solde - laValeur > 0 :
4         self.__solde = self.__solde - laValeur
5     else:
6         print("Compte bloque !!!")
```



En python,

→ le préfixe " `--` " rend privé un attribut ou une méthode

Contrôle d'accès : intérêt

① protéger

→ ne pas permettre l'accès à tout dès que l'on a une référence de l'objet

② masquer l'implémentation

→ toute la décomposition du problème n'a besoin d'être connue du programmeur client

③ évolutivité

→ possibilité de modifier tout ce qui n'est pas public sans impact pour le programmeur client

Contrôle d'accès : intérêt

① protéger

→ ne pas permettre l'accès à tout dès que l'on a une référence de l'objet

② masquer l'implémentation

→ toute la décomposition du problème n'a besoin d'être connue du programmeur client

③ évolutivité

→ possibilité de modifier tout ce qui n'est pas public sans impact pour le programmeur client

Contrôle d'accès : intérêt

① protéger

→ ne pas permettre l'accès à tout dès que l'on a une référence de l'objet

② masquer l'implémentation

→ toute la décomposition du problème n'a besoin d'être connue du programmeur client

③ évolutivité

→ possibilité de modifier tout ce qui n'est pas public sans impact pour le programmeur client

Règles

Règles à respecter

- ➊ Rendre **privés les attributs** caractérisant l'état de l'objet
- ➋ Fournir des *méthodes publiques* permettant de accéder/modifier l'attribut
 - accesseurs
 - mutateurs

1 - Définir les classes

```
1 #####
2 # Programme Python          #
3 # Auteur : .....          #
4 # Date creation : ...      #
5 #####
6
7 class X:
8
9     # --- Documentation ---
10
11
12     # --- Constructeur et destructeur ---
13
14     # --- Methodes publiques: Acces aux attributs ---
15         # accesseurs
16         # mutateurs
17
18     # --- Methodes publiques: Operations ---
19
20     # --- Methodes privees ---
```

1 - Définir les classes

Documentation

- Conception :
 - décrire la classe et ses fonctionnalités
 - Debuggage :
 - débayer des parties de code, marquer des points douteux, signer un correctif
 - Maintenance :
 - décrire les corrections/ajouts/retraits et les signer
- **Indispensable** dans un code
- description, pas des "faut que j'aille prendre un café" ...

1 - Définir les classes

Constructeur et destructeur

- **Constructeur**

- Définir l'état initial de l'objet
 - Initialisation des attributs définis dans la classe
- Créer les objets dont l'existence est liée à celle de l'objet
- Types de constructeur
 - par défaut, par copie ...

- **Destructeur**

- Détruire tous les objets créés par l'objet courant au cours de son existence

1 - Définir les classes

Accès aux attributs : `getAttrName` et `setAttrName`

- **Objectifs**
 - Maintien de l'intégrité
 - Limitation des effets de bord
- **Comment ?**
 - Attribut : toujours un membre **privé**
 - `getAttrName` et `setAttrName` : en fonction des contraintes d'intégrité
- **Exemples**
 - Taille d'une Liste
 - Dimensions d'un Rectangle et d'un Carré

1 - Définir les classes

```
1  class Temperature:
2      """
3      La classe Temperature represente une grandeur physique liee
4      a la notion immediate de chaud et froid
5      """
6      # --- Constructeur et destructeur
7      def __init__(self, laValeur = 0):
8          self.__valeur = laValeur
9
10     def __del__(self):
11         print "destruction"
12
13     # --- Methodes publiques: Acces aux attributs
14         # accesseurs
15     def getValeur(self):
16         return self.__valeur
17         # mutateurs
18     def setValeur(self, value):
19         self.__valeur = value
20
21     # --- Methodes publiques: Operations
22     def getValeurC(self):
23         return self.__valeur - 273.16
24
25     def getValeurF(self):
26         return 1.8 * self.getValeurC() + 32
```

2 - Instancier des objets et les utiliser

```
1  # Exemple d'utilisation de cette classe
2
3  eauChaude = Temperature(310)
4  print eauChaude.getValeurC()
5  print eauChaude.getValeurF()
6
7  eauFroide = Temperature(5)
8  print eauFroide.getValeurC()
9  print eauFroide.getValeurF()
```

Intérêt

```
1  class Voiture :
2      """
3      La classe Voiture definie par sa largeur
4      """
5      # --- Constructeur et destructeur
6      def __init__(self, laValeur = 0):
7          self.__largeur = laValeur
8
9      # --- Methodes publiques: Acces aux attributs
10         # accesseurs
11         def getLargeur(self):
12             return self.__largeur
13         # mutateurs
14         def setLargeur(self, value):
15             self.__largeur = value
16
17         # --- Methodes publiques: Operations
18         def mettreAJour(self):
19             ...
20
21         ##### Objet #####
22         vehicule1 = Voiture();
23         vehicule1.setLargeur(100) # Largeur de la voiture
24         vehicule1.mettreAJour()   # Mettons a jour l'affichage
25         vehicule1.setLargeur(150) # Changeons la largeur de la voiture
26         vehicule1.mettreAJour()   # Mettons a jour l'affichage
```

Intérêt

```
1     def setLargeur(self, value):
2         self.__largeur = value
3         self.mettreAJour()
4
5     ##### Objet #####
6     vehicule1 = Voiture();
7     vehicule1.setLargeur(100) # Largeur de la voiture
8     # L'affichage de la voiture est automatiquement mis a jour dans l'accesseur
9     vehicule1.setLargeur(150) # Changeons la largeur de la voiture
10    # Encore une fois, il n'y a rien d'autre a faire !
```

Intérêt

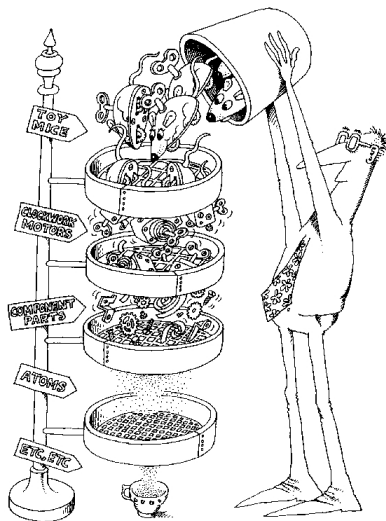
```
1  vehicule1 = Voiture();
2  vehicule1.setLargeur(100) # Largeur de la voiture
3  # On verifie que la largeur est dans les limites
4  if vehicule1.getLargeur() < 100 : vehicule1.setLargeur(100)
5  else if vehicule1.getLargeur() > 200 : vehicule1.setLargeur(200)
6  print vehicule1.getLargeur() # Affiche: 100
7
8  vehicule1.setLargeur(250) # Changeons la largeur de la voiture
9  if vehicule1.getLargeur() < 100 : vehicule1.setLargeur(100)
10 else if vehicule1.getLargeur() > 200 : vehicule1.setLargeur(200)
11 print vehicule1.getLargeur() # Affiche: 200
```

Intérêt

```
1  def setLargeur(self, value):
2      self.__largeur = value
3
4      # _largeur doit etre comprise entre 100 et 200
5      if(self.getLargeur() < 100) :          self.__largeur = 100
6      else if(self.getLargeur() > 200) : self.__largeur = 200
7
8      self.mettreAJour()
9
10 ### Objet
11 vehicule1 = Voiture();
12 vehicule1.setLargeur(100) # Largeur de la voiture
13 # Plus besoin de verifier que la largeur est dans les limites , l'acceseur le
14 print vehicule1.getLargeur() # Affiche: 100
15
16 vehicule1.setLargeur(250); # Changeons la largeur de la voiture
17 print vehicule1.getLargeur() # Affiche: 200
```

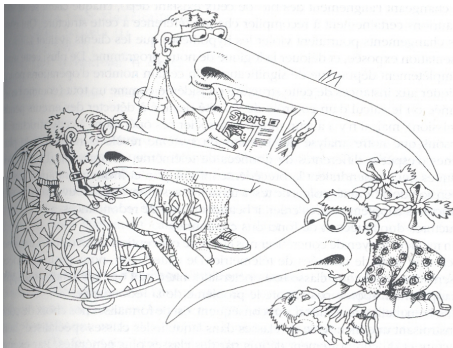
Plan de la présentation

- 1 Justification du modèle objet (1 UC)
- 2 Concepts fondateurs (1 UC)
- 3 Encapsulation (1 UC)
- 4 Hiérarchie (1 UC)**
- 5 Collaborations entre objets (1 UC)
- 6 Pour aller plus loin



Les entités constituent une hiérarchie [Booch, 1992]

Héritage



Une sous-classe peut hériter de la structure et du comportement de sa superclasse (classe mère) [Booch, 1992]

Héritage

Exemple : Fichier

- Il existe plusieurs sortes de fichier :
 - fichier binaire → `FichierBinaire`
 - fichier ASCII → `FichierASCII`

- Certaines propriétés sont *générales* à toutes les classes
 - `nom`, `taille`
- Certaines sont *spécifiques* à une classe ou un groupe de classes spécialisées
 - `executer`, `imprimer`

Héritage

La relation d'héritage

- **Principe**

les caractéristiques des classes supérieures sont héritées par les classes inférieures.

- **Mise en œuvre**

les connaissances les plus générales sont mises en commun dans des classes qui sont ensuite *spécialisées* par définitions de sous-classes successives contenant des connaissances de plus en plus spécifiques.

Spécialisation

La spécialisation d'une classe peut être réalisée selon deux techniques :

- ① l'*enrichissement* : la sous-classe est dotée de nouvelle(s) méthode(s) et/ou de nouveau(x) attribut(s) ;
- ② la *surcharge* : une (ou plusieurs) méthode et/ou un attribut hérité est redéfini.

En pratique ...

Enrichissement (ajout d'une méthode)

```
1  #####
2  ##### class Fichier #####
3
4  class Fichier:
5      """ Definition classe Fichier """          # Documentation
6
7      def __init__(self):                          # Constructeur
8          print "creation d'un fichier"
9
10
11 #####
12 ##### class FichierBinaire #####
13
14 class FichierBinaire(Fichier):
15     """ Definition classe FichierBinaire """    # Documentation
16
17     def __init__(self):                          # Constructeur
18         Fichier.__init__(self)
19
20     def imprimer(self):                          # Enrichissement
```

En pratique ...

Enrichissement (ajout d'un attribut)

```

1  #####
2  ##### class Fichier #####
3
4      class Fichier:
5          """ Definition classe Fichier """           # Documentation
6
7          def __init__(self):                             # Constructeur
8              print "creation d'un fichier"
9
10
11 #####
12 ##### class FichierBinaire #####
13
14      class FichierBinaire(Fichier):
15          """ Definition classe FichierBinaire """     # Documentation
16
17          def __init__(self):                             # Constructeur
18              Fichier.__init__(self)
19              self.__codage=32                             # Enrichissement

```

En pratique ...

Surcharge d'un attribut

```
1 #####
2 ##### class Fichier #####
3
4 class Fichier:
5     def __init__(self):                # Constructeur
6         self.nom = "fichier_sans_nom"
7
8
9 #####
10 ##### class FichierBinaire #####
11
12 class FichierBinaire(Fichier):
13
14     def __init__(self):                # Constructeur
15         Fichier.__init__(self)
16         self.nom = "fichierBinaire_sans_nom" # Surcharge de l'attribut "n
```

En pratique ...

Surcharge d'une méthode

```
1 #####
2 ##### class Fichier #####
3 class Fichier:
4
5     def __init__(self):                                # Constructeur
6         print "creation d'un fichier"
7
8     def ouvrir(self):
9         print "ouvrir fichier"
10
11
12 #####
13 ##### class FichierBinaire #####
14 class FichierBinaire(Fichier):
15
16     def __init__(self):                                # Constructeur
17         Fichier.__init__(self)
18
19     def ouvrir(self):                                  # Surcharge de la methode
20         print "ouvrir fichier Binaire"                # "ouvrir"
```

En pratique ...

```
1  class Engin:
2      def __init__(self, braquage=90):
3          self.__vitesse = 0
4          self.__braquage = braquage
5
6      def virage(self):
7          return self.__vitesse / self.__braquage
8
9      # -----
10 class Sous_marin_alpha(Engin):
11     def __init__(self, braquage):
12         Engin.__init__(self, braquage)
13
14     # -----
15 class Torpille(Engin):
16     def __init__(self, braquage):
17         Engin.__init__(self, braquage)
18
19     def explode(self):
20         print "Explosion !!!!!!"
```

L'héritage multiple

- faire hériter une classe de plusieurs superclasses.
- regrouper au sein d'une seule et même classe les attributs et méthodes de plusieurs classes.

En pratique ...

```
1
2  class Herbivore:
3      ...
4      def mangerHerbe(self):
5          print "je mange de l'herbe"
6
7  #-----
8  class Carnivore:
9      ....
10     def mangerViande(self):
11         print "je mange de la viande"
12
13  #-----
14  class Omnivore(Herbivore, Carnivore):  # Heritage multiple
15     ....
16     def manger(self):
17         if rand(10)%2 == 0 :
18             self.mangerHerbe()
19         else:
20             self.mangerViande()
```

Polymorphisme

- A un **même service** peut correspondre, dans les classes dérivées, des **méthodes différentes**
- Chaque classe dérivée définit le *forme* (**-morphe**) sous laquelle elle remplit le service (méthode)
- Cette forme peut donc être *multiple* (différente d'une classe à l'autre) (**poly-**)

En pratique ...

```
1  class Shape:                                # classe "abstraite" specifiant une interface
2      def __init__(self):
3          print "je suis une shape"
4
5      def area(self):
6          return "shape area"
7
8  class Rectangle(Shape):
9      def __init__(self):
10         Shape.__init__(self)
11         print "je suis un Rectangle"
12
13
14         def area(self):
15             return "rectangle area"
16
17  class Circle(Shape):
18      def __init__(self):
19         Shape.__init__(self)
20         print "je suis un cercle"
21         def area(self):
22             return "cirle area"
23
24  ##### programme principal
25  shapes = []
26  shapes.append(Rectangle())
27  shapes.append(Circle())
28  for i in shapes:
```

Retour sur l'encapsulation

Encapsulation : contrôle d'accès

- publiques
- privées
- **protégées** : les attributs ne sont visibles que des sous-classes

Retour sur l'encapsulation

Encapsulation : règles d'écriture

- publiques : `attr1`, `attr2`
- privées : `__attr1`, `__attr2`
- **protégées** : `_attr1`, `_attr2`



En python,

→ Les attributs et méthodes **protégées** sont accessibles normalement, le préfixe `_` pour seul objectif d'informer sur leur nature

En pratique ...

```
1 class Telephone:
2     def __init__(self):
3         # donnees privées
4         self.__numero_serie = '126312156 '
5
6         # donnees protegees
7         self._code_pin = '1234 '
8
9         # donnees publiques
10        self.modele = 'nokia 6800 '
11        self.numero = '06 06 06 06 06 '
```

En pratique ...

```
1         # methodes privees
2
3         # methodes protegees
4         def _chercherReseau(self):
5             print 'Reseau FSR, bienvenue dans un monde meilleur ...'
6
7         def _recupMessage(self):
8             print 'Pas de message'
9
10        # methodes publiques
11        def allumer(self, codePin):
12            print self.modele
13            if self._code_pin == codePin :
14                print 'Code pin OK'
15                self._chercherReseau()
16                self._recupMessage()
17            else:
18                print 'mauvais code pin'
```

En pratique ...

```
1
2  # programme principal
3  nokia = Telephone()
4  nokia.allumer( '1524 ' )
5  nokia.allumer( '1234 ' )
```

En pratique ...

```
1  class TelephonePhoto(Telephone):
2      def __init__(self):
3          Telephone.__init__(self)
4          print 'telephone + mieux'
5
6      # methodes publiques
7      def prendre_photo(self):
8          print 'clik-clak'
9
10     # Test
11     def fonction_test(self):
12         print self.__numero_serie # ERREUR
13         print self._code_pin      # OK sous-classe de Telephone
14
15
16     # programme principal
17     nokia = TelephonePhoto()
18     nokia.allumer('1234')
19     nokia.prendre_photo()
```

En pratique ...

```
1  tel_sagem = Telephone()
2  print    tel_sagem._numero_serie    # ERREUR
3  print    tel_sagem._code_pin        # NON !!
```

En pratique ...

```
1  # Relation multiple
2
3  class FilleMars:
4      def __init__(self, prenom):
5          self.__prenom = prenom
6
7      def affichePrenom(self):
8          print self.__prenom
9  # -----
10 class PapaMars:
11     def __init__(self):
12         self.__filles = []
13         self.__filles.append( FilleMars( 'Pamela' ) )
14         self.__filles.append( FilleMars( 'Tina' ) )
15         self.__filles.append( FilleMars( 'Rebecca' ) )
16         self.__filles.append( FilleMars( 'Amanda' ) )
17
18     def cri_a_table(self):
19         for child in self.__filles:
20             child.affichePrenom()
21         print ( 'a table !!! ' )
```

Attention aux pièges

Héritage vs associations

classe `Automobile` à partir des classes `Carrosserie`, `Siege`, `Roue` et `Moteur`.

- l'héritage multiple == ERREUR

En pratique ...

```
1  # Relation simple
2  class Calculateur():
3      """ classe de calculs """
4      def somme(self, args = [])
5          resultat = 0
6          for arg in args:
7              resultat += arg
8          return resultat
9  # -----
10 class Afficheur():
11     """ classe de gestion de l'interface """
12     def __init__(self):
13         self.calculateur = Calculateur()
14
15     def somme(self, args = []):
16         resultat = self.calculateur.somme(args)
17         print 'le resultat est %d' % resultat
```

Association simple

Un objet peut faire appel aux compétences d'un autre objet

Objet

flic_Wiggum



arrêter →



Objet homer_mobile

Association simple

Un objet peut faire appel aux compétences d'un autre objet

Objet

flic_Wiggum



arrêter →



Objet homer_mobile

Comportement ... En pratique

```
1  ##### definition classe #####
2  class Flic:
3      def __init__(self):
4          self.nom
5
6      def donnerOrdreStop(self, vehicule): ## instance de Voiture
7          print "arrete toi !!"
8          vehicule.arreter()
9
10 ##### Objet #####
11 homer_mobile = Voiture()
12 flic_Wiggum = Flic()
13 flic_Wiggum.donnerOrdreStop(homer_mobile)
```

En pratique ...

```
1  class Roue:
2      def __init__(self):
3          print "Me voici !"
4
5      def __del__(self):
6          print "Je meurs !"
7
8
9  class Voiture:
10     def __init__(self):
11         self.roue1 = Roue()
12         self.roue2 = Roue()
13         self.roue3 = Roue()
14         self.roue4 = Roue()
15
16     def __del__(self):
17         print "I'm dying!"
18         del self.roue1
19         del self.roue2
20         del self.roue3
21         del self.roue4
```

En pratique ...

```
1  class Humain:
2      def __init__(self, nom):
3          self.__nom = nom
4
5  class Roue:
6      def __init__(self):
7          print "Me voici !"
8
9      def __del__(self):
10         print "Je meurs !"
11
12 class Voiture:
13     def __init__(self, proprio):
14         self.__roue1 = Roue()
15         self.__roue2 = Roue()
16         self.__roue3 = Roue()
17         self.__roue4 = Roue()
18         self.__conducteur = proprio("")
19
20     def __del__(self):
21         print "I'm dying!"
22         del self.__roue1
23         del self.__roue2
24         del self.__roue3
25         del self.__roue4
```

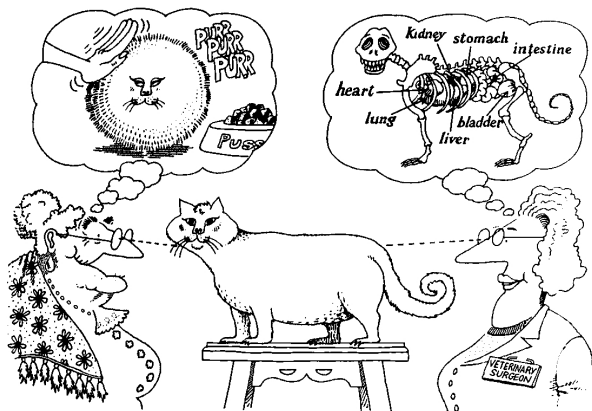
Plan de la présentation

- 1 Justification du modèle objet (1 UC)
- 2 Concepts fondateurs (1 UC)
- 3 Encapsulation (1 UC)
- 4 Hiérarchie (1 UC)
- 5 Collaborations entre objets (1 UC)
- 6 Pour aller plus loin

Cours 5 - Programmation objet avancée

Objectifs

- Abstraction
- Classification
- Modularité
- Typage
- Simultanéité
- Persistance



L'abstraction se concentre sur les caractéristiques essentielles d'un objet, selon le point de vue de l'observateur [Booch, 1992]

Abstraction

- Un service peut être déclaré sans qu'une méthode y soit associée : **service abstrait**
- La méthode associée au service est définie dans une ou plusieurs classes dérivées
- On ne peut pas instancier d'objets d'une classe possédant un service abstrait : **classe abstraite**



En python, attention, on peut instancier une classe abstraite

En pratique ...

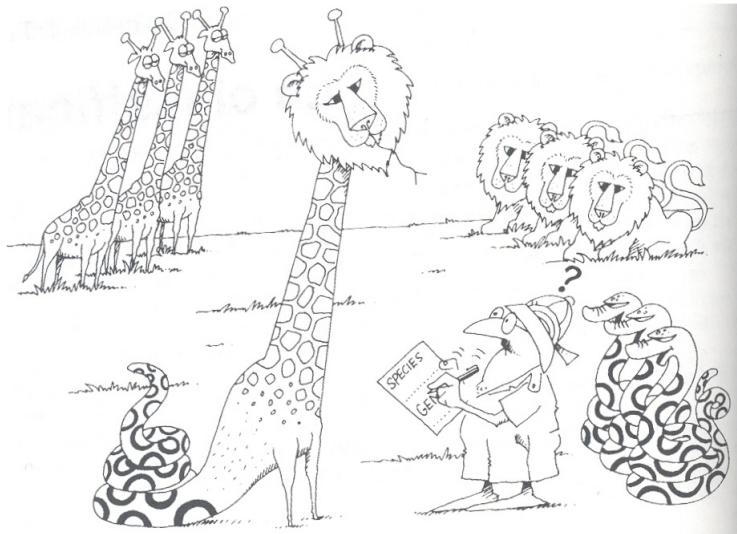
```
1
2  class AbstractClass :
3
4      ...
5
6      # Force la classe fille a definir cette methode
7      def getValue(self):
8          raise NotImplementedError()
9
10     # methode commune
11     def printOut(self):
12         print self.getValue();
```

En pratique ...

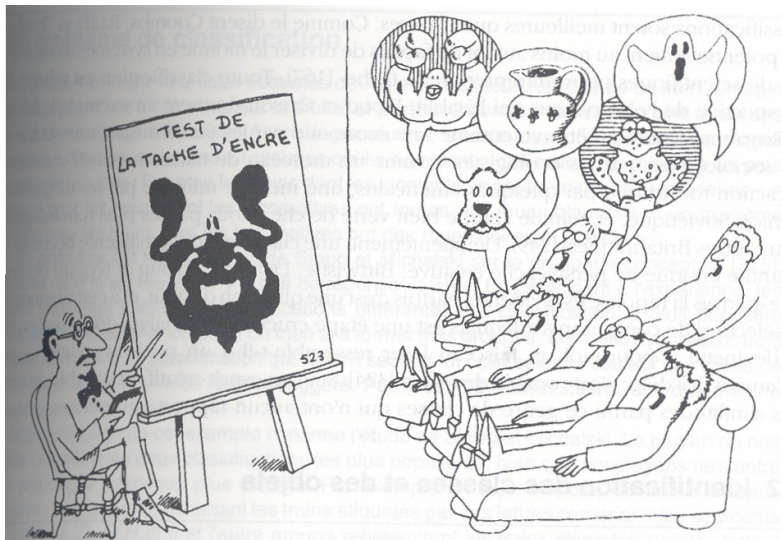
```
1  class ConcreteClass1( AbstractClass ) :
2
3      ...
4
5      def getValue( self ):
6          return " ConcreteClass1 "
7
8  class ConcreteClass2( AbstractClass ) :
9
10     ...
11
12     def getValue( self ):
13         return " ConcreteClass2 "
14
15     ##### Programme principal #####
16     class1 = ConcreteClass1 ()
17     class1.printOut ()
18
19     class2 = ConcreteClass2 ()
20     class2.printOut ()
```



Les classes et les objets doivent être au juste niveau d'abstraction : ni trop haut ni trop bas [Booch, 1992]

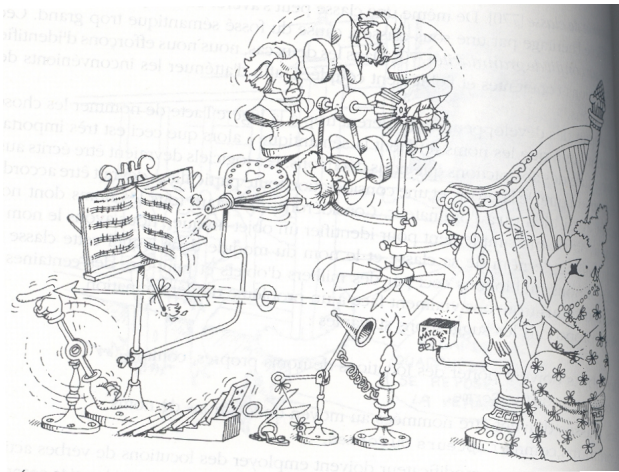


La classification est le moyen par lequel nous ordonnons nos connaissances [Booch, 1992]

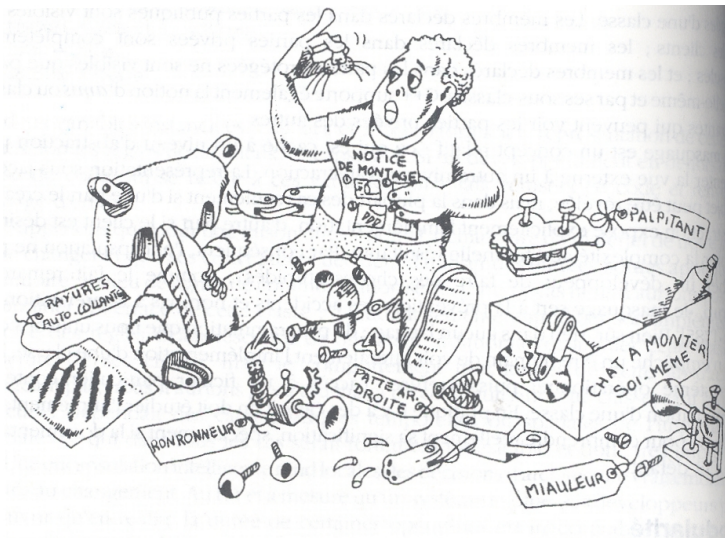


Des observateurs différents classent le même objet de différentes façons
[Booch, 1992]

Identification des mécanismes



Les mécanismes sont le moyen par lequel les objets collaborent pour procurer un certain comportement de plus haut niveau [Booch, 1992]

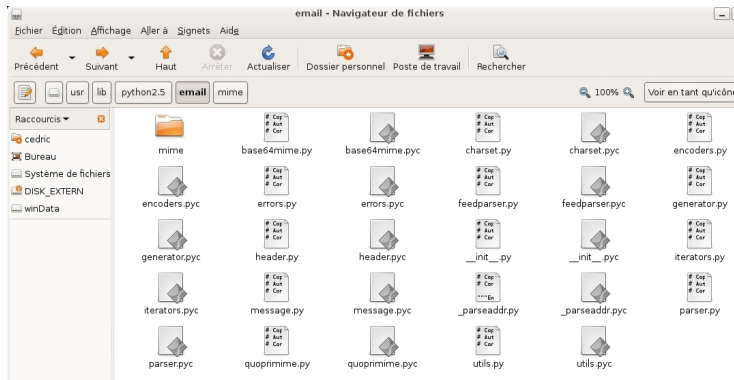


La modularité regroupe les entités en unités discrètes [Booch, 1992]

En pratique ...

Paquet en python

- répertoire avec un fichier `__init__.py` qui définit les modules qu'il contient (exemple : `/usr/lib/python/xml/`)
- **from paquetage**



En pratique ...

Module en python

- un fichier regroupant variables, classes et fonctions
- extension ".py"
- `from paquetage import Module`
- attention la liste `path` de `sys`
 - `sys.path.append(Module)`

En pratique ...

Classe dans un module en python

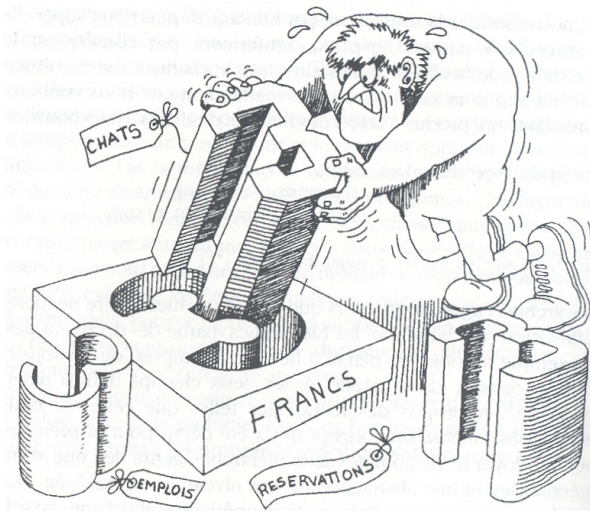
- Une classe
 - `from paquetage import Class`
 - `from turtle import Pen`
`p1 = Pen()`

Typage

Chaque objet peut être typé

Le type définit

- la syntaxe (comment l'appeler ?)
- la sémantique (qu'est ce qu'il fait ?)



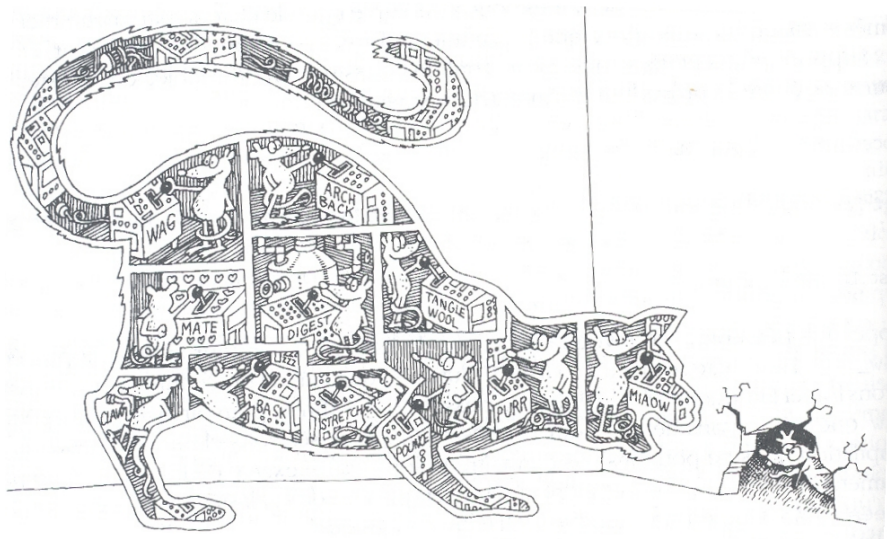
Un typage fort empêche le mélange des entités [Booch, 1992]

En pratique ...

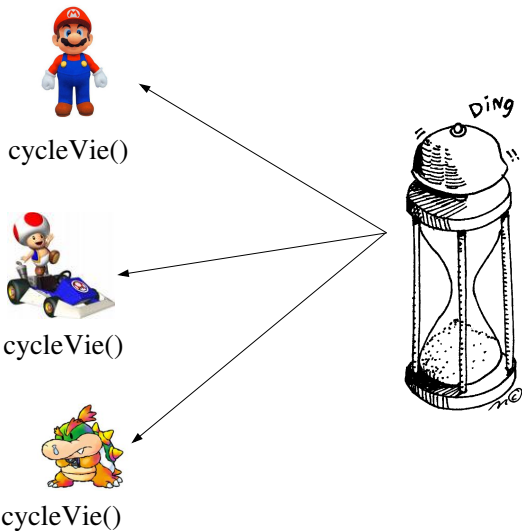
```
1  def estPair(nombre):  
2      if not isinstance(nombre, int):  
3          return None  
4      return (nombre % 2) == 0
```



En python, `isinstance(X,type)` permet de vérifier le type d'un élément



La simultanéité permet à des objets différents d'agir en même temps
[Booch, 1992]



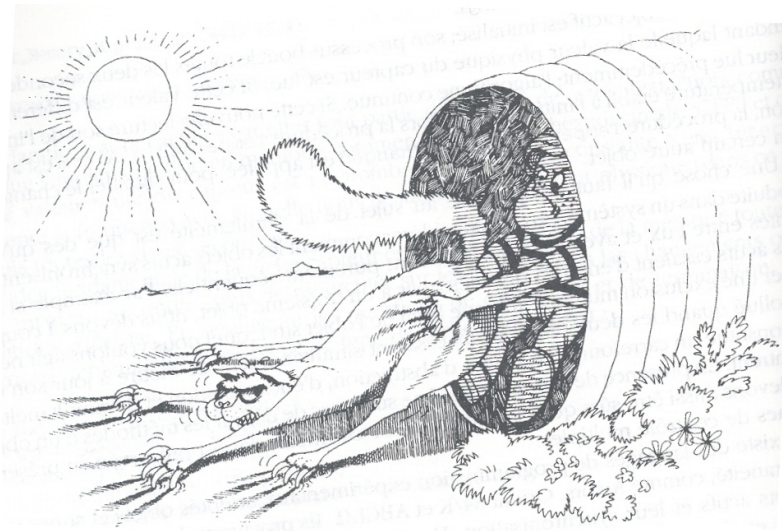
Ordonnanceur d'activités : *class MyTimer*

En pratique ...

```
1  import threading
2  import time
3
4  class MyTimer:
5      def __init__(self, tempo, targets = []):
6          self.__targets = targets
7          self.__tempo = tempo
8
9      def start(self):
10         self.__timer = threading.Timer(self.__tempo, self.__run)
11         self.__timer.start()
12
13     def stop(self):
14         self.__timer.cancel()
15
16     def __run(self):
17         for i in self.__targets:
18             i.cycleVie()
```

En pratique ...

```
1 #####
2 #  Programme principal      #
3 #####
4
5     bots.append( Animat2D() )
6     bots.append( Animat2D() )
7
8     a = MyTimer(1,bots)
9     a.start()
```



La persistance préserve l'état et la classe d'un objet à travers le temps
et l'espace [Booch, 1992]

Persistance par sérialisation

Sauvegarde de l'état d'un objet

- sauvegarder la valeur des attributs de chaque objet
- à partir de ces valeurs, on peut reconstruire l'objet



En python, `--dict--` permet de récupérer les valeurs des attributs d'un objet

En pratique ...

```
1 class Dummy:
2     def __init__(self):
3         self.member1="abcd"
4         self.member2=0
5         self.member3=['a','e','i','o','u']
6
7
8 d1 = Dummy()
9 print d1.__dict__
10
11
12
13 ##### Test #####
14 >>> python testDic.py
15 {'member1': 'abcd', 'member3': ['a', 'e', 'i', 'o', 'u'], 'member2': 0}
```

En pratique ...

Persistance de toutes les instances de classes dérivées d'une classe mère.

```
1  import shelve
2  import atexit
3  import sys
4
5  donnees = None
6  instances = []
7
8  def _charge_objets():
9      print 'chargement ... '
10     global donnees
11     donnees = shelve.open('objets.bin')
12
13  def _sauvegarde():
14      print 'sauvegarde ... '
15      global donnees
16      for instance in instances:
17          donnees[instance.getId()] = instance.__dict__
18      if donnees is not None :
19          donnees.close()
```

En pratique ...

```
1  # chargement des objets
2  _charge_objets()
3
4  # _sauvegarde sera appelee a la fermeture du programme
5  atexit.register(_sauvegarde)
```

En pratique ...

```
1  class Persistent:
2
3      def __init__(self, id):
4          global donnees
5          if id in donnees:
6              self.__dict__ = donnees[id]
7          else:
8              self._id = id
9              instances.append(self)
10
11     def getId(self):
12         return self._id
13
14
15  class MaClasse(Persistent):
16
17     def __init__(self, id, datas = None):
18         Persistent.__init__(self, id)
19         if datas is not None:
20             self.datas = datas
```

En pratique ...

Utilisation

```
1
2  # programme principal
3  instance_de = MaClasse('1')
4  while True:
5      try:
6          if hasattr(instance_de, 'datas'):
7              instance_de.datas = raw_input('valeur (actuelle:%s): ' % instance_de
8          else:
9              instance_de.datas = raw_input('nouvelle valeur :')
10
11  except KeyboardInterrupt:      # par exemple "Control-C"
12      print 'fin '
13      sys.exit(0)
```

En pratique ...

Execution

```
1 >>> python testPersistence.py
2 chargement...
3 nouvelle valeur: homer
4 valeur (actuelle:homer): fin
5 sauvegarde...
6
7 >>> python testPersistence.py
8 chargement...
9 valeur (actuelle:homer): bart
10 valeur (actuelle:bart): fin
11 sauvegarde...
12
13
14 >>> python testPersistence.py
15 chargement...
16 valeur (actuelle:bart): lisa
17 valeur (actuelle:lisa): fin
18 sauvegarde...
```

Références



Booch, G. (1992).

Conception orientée objets et applications.

Addison-Wesley.



Brooks, F. (1987).

No silver bullet - essence and accidents of software engineering.

IEEE Computer, 20(4) :10–19.



Peter, L. (1986).

The peter pyramid.

New York, NY :William Morrow.

Programmation Orientée Objet

— Concepts —

Cédric Buche

buche@enib.fr www.enib.fr/~buche

École Nationale d'Ingénieurs de Brest (ENIB)

28 janvier 2014

